# Contrato de Interfaz e Integración de Servicios

*CampusStudy — Entrega consolidada por el equipo Frontend*

*Samuel Esteban Rodriguez Vargas-Mariana Castillo Acosta- Julian Camilo Velasco*

## A. Información general

| Campo | Valor |
|---|---|
| Proyecto | CampusStudy — Plataforma distribuida para la organización de actividades académicas |
| Servicio que entrega este documento | Frontend (integrador y consumidor de todos los microservicios) |
| Equipo | Equipo Frontend |
| Integrantes | [Nombre integrante 1], [Nombre integrante 2] |
| Arquitectura del sistema | Microservicios, comunicación vía API REST, formato JSON |
| Versión del contrato | 1.0 |
| Versión de la API | /api/v1/ |
| Fecha | 24 de septiembre de 2026 |

## B. Propósito de este documento

El Frontend de CampusStudy no administra datos propios: su responsabilidad es consumir e integrar la información de todos los microservicios del proyecto (usuarios, materias, tareas, grupos, notificaciones y estadísticas) y presentarla de forma coherente al estudiante. Por esta razón, este documento reúne, en un solo lugar, el contrato de cada servicio con el que el Frontend se integra: el JSON real acordado por cada equipo proveedor, y el schema Zod que el Frontend usa para validarlo en tiempo de ejecución. Esta entrega cubre los cuatro servicios ya confirmados (usuarios, materias, tareas, estadísticas); grupos y notificaciones se agregarán cuando sus equipos entreguen su contrato.

## C. Servicios relacionados (vista general)

| Nuestro servicio | Otro servicio | ¿Qué necesitamos? | ¿Quién consume? |
|---|---|---|---|
| Frontend | usuarios-service | Autenticación, datos del usuario | Frontend |
| Frontend | materias-service | Materias del estudiante | Frontend |
| Frontend | tareas-service | Tareas por materia/grupo | Frontend |
| Frontend | estadisticas-service | Métricas de desempeño | Frontend |
| Frontend | grupos-service | Grupos y sus integrantes (pendiente) | Frontend |
| Frontend | notificaciones-service | Alertas del estudiante (pendiente) | Frontend |

Adicionalmente, los servicios de backend se consumen entre sí (por ejemplo, tareas-service valida contra materias-service y usuarios-service, y estadisticas-service consulta a los cuatro). Esas integraciones quedan documentadas dentro de la sección de cada servicio.

## D. Matriz de integración

| Proveedor | Consumidor | Endpoint | Información |
|---|---|---|---|
| usuarios-service | Frontend | POST /api/v1/auth/login | Token de sesión |
| usuarios-service | Frontend | POST /api/v1/auth/registro | Cuenta creada |
| usuarios-service | Frontend | POST /api/v1/auth/validate | Validez del token |
| materias-service | Frontend | GET /api/v1/materias | Mis materias |
| materias-service | tareas-service | GET /api/v1/materias/{materia_id} | Existencia de la materia |
| tareas-service | Frontend | GET /api/v1/tareas | Tareas del estudiante |
| tareas-service | estadisticas-service | GET /api/v1/tareas/resumen | Conteos por estado |
| usuarios-service | estadisticas-service | GET /api/v1/usuarios/{id} | Datos básicos del usuario |
| materias-service | estadisticas-service | GET /api/v1/materias?usuario_id= | Materias inscritas (en conflicto, ver sección E) |
| estadisticas-service | Frontend | GET /api/v1/estadisticas/usuario/{id} | Reporte de desempeño |

## E. Discrepancias detectadas — pendientes de resolver en la reunión de integración

| Punto | Detalle |
|---|---|
| Formato de respuesta (envelope) | Cinco estilos distintos entre los seis servicios: Tareas envuelve en {exito, datos, mensaje}; Materias y Grupos responden el objeto plano; Usuarios responde plano pero con su propio {error, errors[]} para fallos; Estadísticas no define un formato de error explícito; Notificaciones usa {codigo, mensaje, ...} incluso en respuestas exitosas. |
| Nombre del correo electrónico | usuarios-service usa email; el resto del proyecto (y el diseño original) usaba correo. Hay que decidir un único nombre. |
| Filtro de materias por usuario | estadisticas-service espera GET /api/v1/materias?usuario_id={id}; el contrato actual de materias-service solo expone "mis materias" vía JWT, sin filtro por usuario ajeno. |
| Rol de usuario | usuarios-service expone un campo rol. El diseño original de CampusStudy asumía una app sin roles (solo estudiantes). Confirmar si el Frontend debe usarlo. |
| Estados de tarea | tareas-service define 4 estados reales (pendiente/en_progreso/completada/cancelada) más "vencida" como conteo derivado. estadisticas-service solo contempla 3 (pendiente/completada/vencida) y no tiene en_progreso ni cancelada en su resumen. |
| Formato de expira_en (login) | usuarios-service lo tipa como number O string sin aclarar cuál — puede ser timestamp Unix o fecha ISO. El Frontend no puede formatear una fecha de expiración sin saber el formato exacto. |
| Procedencia del contrato de Grupos | El JSON Schema recibido no aclara si es la versión oficial confirmada por ese equipo o una propuesta basada en trabajo previo del |

| | |
|---|---|
| | Frontend. Además, no llegaron secciones de errores, reglas de fallo, versionamiento ni pruebas de integración — solo la forma de los datos. |
| Notificaciones — significado de "estado" (contradicción interna) | La sección B de su propio documento dice que indica "si está pendiente o fue leída"; la sección D y todos los ejemplos JSON usan valores como "realizado" (éxito/fallo de la operación, no de lectura). El equipo no puede tener las dos cosas a la vez. |
| Notificaciones — hora de creación (contradicción interna) | La sección B dice que administran "hora de creación" como dato aparte; la tabla de datos (sección D) y todos los ejemplos JSON no tienen ningún campo de hora, solo fecha_creacion (YYYY-MM-DD). |
| Notificaciones — referencia genérica | Usan un único referencia_id (integer) para apuntar a la tarea, grupo o materia que originó el aviso, en vez de campos específicos. El Frontend tendría que inferir el tipo de recurso a partir del campo tipo (string libre, sin lista cerrada de valores) para saber a qué apunta ese id. |
| Notificaciones — ejemplo de GET incompleto | El ejemplo de respuesta de GET /notification/{usuario_id} en su documento solo trae tipo, titulo, mensaje y estado — le faltan notificacion_id, usuario_id, fecha_creacion y referencia_id que sí exige la tabla de datos de la sección D. No se puede confirmar cuál es la forma real hasta que el equipo lo aclare. |

# F. Contrato: usuarios-service (autenticación)

## F.1 Propósito del servicio

Administra el registro, autenticación y validación de sesión de los estudiantes. Es el único servicio que el Frontend consume sin necesidad de un token previo (login y registro).

## F.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| Login / registro / validación de token | usuarios-service | Frontend |
| Datos básicos del usuario | usuarios-service | estadisticas-service |

## F.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| POST | /api/v1/auth/login | Iniciar sesión y obtener el token. |
| POST | /api/v1/auth/registro | Crear una cuenta nueva. |
| POST | /api/v1/auth/validate | Validar un token existente. |

## F.4 Solicitud y respuesta

*Nota: el documento entregado por usuarios-service incluyó el código Zod y un ejemplo de cliente HTTP, pero no un JSON de ejemplo explícito. El siguiente ejemplo es ilustrativo, construido a partir de ese schema.*

```
// Solicitud: POST /api/v1/auth/login
{
  "email": "ana@uniminuto.edu.co",
  "password": "12345678"
}

// Respuesta 200 OK
{
  "usuario_id": 25,
  "nombre": "Ana",
  "email": "ana@uniminuto.edu.co",
  "rol": "estudiante",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "tipo_token": "Bearer",
  "expira_en": 3600
}
```

| Campo | Tipo | Obligatorio | Descripción |
|---|---|---|---|
| usuario_id | integer | Sí | Identificador del usuario. |
| nombre | string | Sí | Nombre del estudiante. |
| email | string (email) | Sí | Correo del estudiante. |
| rol | string | Sí | Rol del usuario (valores exactos sin confirmar). |
| token | string | Sí | Token de sesión (JWT). |
| tipo_token | string | Sí | Ej. "Bearer". |

| | | | |
|---|---|---|---|
| expira_en | number \| string | Sí | Vigencia del token (formato sin confirmar). |

## F.5 Errores

*El documento de origen no entregó una tabla de errores como Tareas o Estadísticas; esto se dedujo del código de manejo de errores (apiErrorSchema) y del cliente HTTP de ejemplo que incluyeron.*

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 400 | Datos de entrada inválidos (formato de email, password corta, etc.). | Mostrar los errores de errors[] (campo + mensaje) junto al formulario. |
| 401 | Credenciales inválidas, o token inválido/expirado en /validate. | Redirigir al login. |
| 500 | Error interno del servicio (sin confirmar formato exacto). | Mostrar error genérico; reportar con soporte. |
| 503 | usuarios-service no disponible. | Mostrar "Servicio de autenticación no disponible temporalmente" (tal como lo maneja el ejemplo de cliente HTTP entregado por el equipo). |

## F.6 Reglas de datos

- Identificadores: usuario_id, entero positivo.
- email: formato de correo válido (no confirmado si acepta cualquier dominio o solo institucional).
- password: mínimo 8 caracteres (validado en el registro).
- rol: string — valores permitidos sin confirmar con el equipo.
- fecha_registro: presente pero sin formato confirmado (se recomienda solicitar que sea YYYY-MM-DDTHH:MM:SS, igual que el resto del proyecto).

## F.7 Manejo de fallos

El documento de origen no detalla un procedimiento de fallos como sí lo hace Tareas. Comportamiento mínimo asumido, pendiente de confirmar:

- Si usuarios-service no responde, ningún otro servicio (tareas-service, estadisticas-service) debe asumir que el usuario no existe.
- Los servicios que dependan de usuarios-service deben responder 503 a su propio consumidor, no 401 ni 404, cuando la causa es la falta de respuesta del servicio de autenticación.

## F.8 Versionamiento

No especificado en el documento de origen. Se asume versión 1.0 (borrador) hasta que el equipo lo confirme explícitamente, junto con /api/v1/ como base.

## F.9 Prueba de integración

- 1. Login con credenciales válidas → 200 con token.
- 2. Login con contraseña incorrecta → 401.

- 3. Registro con un email ya usado → confirmar código exacto con el equipo (no está definido en el documento; se propone 409).
- 4. POST /api/v1/auth/validate con un token vencido → valido: false.
- 5. Apagar usuarios-service y confirmar que el Frontend muestra "servicio no disponible" en vez de fallar silenciosamente.

## F.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const usuarioSchema = z.object({
  usuario_id: z.number().int().positive(),
  nombre: z.string().min(1),
  email: z.string().email(),
  rol: z.string(),
  estado: z.string().optional(),
  fecha_registro: z.string().optional(),
});

export type Usuario = z.infer<typeof usuarioSchema>;

export const loginInputSchema = z.object({
  email: z.string().email("Debe ser un email válido"),
  password: z.string().min(8, "La contraseña debe tener al menos 8 caracteres"),
});

export type LoginInput = z.infer<typeof loginInputSchema>;

export const registroInputSchema = z.object({
  nombre: z.string().min(1, "El nombre es obligatorio"),
  email: z.string().email("Debe ser un email válido"),
  password: z.string().min(8, "La contraseña debe tener al menos 8 caracteres"),
});

export type RegistroInput = z.infer<typeof registroInputSchema>;

export const loginDataSchema = z.object({
  usuario_id: z.number().int().positive(),
  nombre: z.string(),
  email: z.string().email(),
  rol: z.string(),
  token: z.string(),
  tipo_token: z.string(),
  expira_en: z.number().or(z.string()),
});

export type LoginData = z.infer<typeof loginDataSchema>;

export const validateInputSchema = z.object({
  token: z.string(),
});

export type ValidateInput = z.infer<typeof validateInputSchema>;

export const validateResponseSchema = z.object({
  valido: z.boolean(),
```

```
  usuario: usuarioSchema.optional(),
});

export type ValidateResponse = z.infer<typeof validateResponseSchema>;

export const errorDetalleSchema = z.object({
  campo: z.string(),
  mensaje: z.string(),
});

export type ErrorDetalle = z.infer<typeof errorDetalleSchema>;

export const apiErrorSchema = z.object({
  error: z.string(),
  errors: z.array(errorDetalleSchema).optional(),
});

export type ApiError = z.infer<typeof apiErrorSchema>;
```

# G. Contrato: materias-service

## G.1 Propósito del servicio

Administra las materias que cada estudiante crea para organizar sus tareas y grupos. Una materia es un espacio de organización propio del estudiante (nombre + NRC) — no es una oferta académica con créditos ni cupos, ya que CampusStudy no gestiona matrícula. Cada materia pertenece a un único estudiante, identificado a través del token JWT.

## G.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| CRUD de materias propias | materias-service | Frontend |
| Validar materia para una tarea | materias-service | tareas-service |
| Materias inscritas por usuario (en conflicto, ver sección E) | materias-service | estadisticas-service |

## G.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| POST | /api/v1/materias | Crear una materia. |
| GET | /api/v1/materias | Listar mis materias (según JWT). |
| GET | /api/v1/materias/{materia_id} | Consultar una materia — usado también por tareas-service. |
| PUT | /api/v1/materias/{materia_id} | Editar nombre/NRC. |
| DELETE | /api/v1/materias/{materia_id} | Eliminar una materia. |

## G.4 Solicitud y respuesta

```
// Solicitud: POST /api/v1/materias
{
  "nombre": "Sistemas Distribuidos",
  "nrc": "12345"
}

// Respuesta 201 Created
{
  "materia_id": 4,
  "nombre": "Sistemas Distribuidos",
  "nrc": "12345",
  "fecha_creacion": "2026-09-20T14:30:00"
}
```

| Campo | Tipo | Obligatorio | Descripción |
|---|---|---|---|
| materia_id | integer | Sí | Identificador único de la materia. |
| nombre | string | Sí | Nombre de la materia (1-100 caracteres). |
| nrc | string | Sí | Número de referencia de curso, definido por el estudiante. |
| fecha_creacion | string (YYYY-MM-DDTHH:MM:SS) | Sí | Momento de creación, hora Colombia. |

## G.5 Errores

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 400 | nombre o nrc ausente o con formato inválido. | Corregir el valor antes de reintentar. |
| 401 | Token ausente o inválido. | Redirigir a login. |
| 404 | La materia no existe, o no pertenece al usuario del token. | No asumir que el id es inválido en general; no completar la operación dependiente. |
| 500 | Error interno de materias-service. | Registrar el error; reintentar una vez. |
| 503 | materias-service temporalmente no disponible. | No asumir que la materia no existe; no crear la tarea/grupo dependiente hasta confirmar. |

## G.6 Reglas de datos

- materia_id: entero positivo.
- nombre: string, 1 a 100 caracteres.
- nrc: string, 1 a 20 caracteres (se mantiene como texto, no como número, para admitir ceros a la izquierda).
- fecha_creacion: YYYY-MM-DDTHH:MM:SS, hora de Colombia (UTC-5), igual convención que tareas-service.
- Convención de nombres: snake_case.

## G.7 Manejo de fallos

- Si materias-service no responde a tareas-service: tareas-service debe responder 503 a su propio consumidor, no asumir que la materia no existe, y no crear la tarea.
- El error debe quedar registrado (log) para diagnóstico.

## G.8 Versionamiento

| Campo | Detalle |
|---|---|
| API base | /api/v1/ |
| Versión actual del contrato | 1.0 |
| Historial | 1.0 — Definición inicial (24 de septiembre de 2026). |

## G.9 Prueba de integración

- 1. Crear una materia desde el Frontend y confirmar 201 con los cuatro campos documentados.
- 2. tareas-service consulta GET /api/v1/materias/{materia_id} con esa materia real y confirma 200.
- 3. Repetir con un id inexistente → 404.
- 4. Detener materias-service y confirmar que tareas-service responde 503 a su frontend en vez de fallar silenciosamente.

## G.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const materiaSchema = z.object({
  materia_id: z.number().int().positive(),
  nombre: z.string().min(1),
  nrc: z.string().min(1),
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}$/),
});

export type Materia = z.infer<typeof materiaSchema>;

export const crearMateriaInputSchema = z.object({
  nombre: z.string().min(1),
  nrc: z.string().min(1),
});

export type CrearMateriaInput = z.infer<typeof crearMateriaInputSchema>;

export const materiasListadoSchema = z.array(materiaSchema);

export const mensajeExitoSchema = z.object({
  mensaje: z.string(),
});

export type MensajeExito = z.infer<typeof mensajeExitoSchema>;
```

# H. Contrato: tareas-service

## H.1 Propósito del servicio

Administra las actividades académicas que los estudiantes deben cumplir dentro de CampusStudy: trabajos, talleres, exposiciones, lecturas y entregas, individuales o de grupo. Permite crearlas, consultarlas, actualizarlas, cambiar su estado y cancelarlas. No guarda datos personales del estudiante, ni el catálogo de materias, ni la conformación de los grupos — solo sus identificadores.

## H.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| CRUD de tareas | tareas-service | Frontend |
| Validar usuario existente y activo | usuarios-service | tareas-service |
| Validar materia existente | materias-service | tareas-service |
| Validar grupo e integrantes | grupos-service | tareas-service |
| Avisar eventos de tareas | tareas-service | notificaciones-service |
| Resumen de tareas por estado | tareas-service | estadisticas-service |

## H.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| POST | /api/v1/tareas | Crear una tarea. |
| GET | /api/v1/tareas/{tarea_id} | Consultar una tarea. |
| GET | /api/v1/tareas?usuario_id=&materia_id=&grupo_id=&estado= | Listar tareas con filtros y paginación. |
| PUT | /api/v1/tareas/{tarea_id} | Actualizar título, descripción, fecha, hora o prioridad. |
| PUT | /api/v1/tareas/{tarea_id}/estado | Cambiar el estado de una tarea. |
| DELETE | /api/v1/tareas/{tarea_id} | Cancelar (eliminación lógica → estado cancelada). |
| GET | /api/v1/tareas/resumen?usuario_id= | Conteos por estado (consumido por estadisticas-service). |

## H.4 Solicitud y respuesta

```
// Solicitud: POST /api/v1/tareas
{
  "titulo": "Contrato de interfaz",
  "descripcion": "Documento del servicio de Tareas",
  "usuario_id": 692464,
  "materia_id": 12,
  "grupo_id": 1,
  "fecha_entrega": "2026-09-23",
  "hora_entrega": "18:00",
  "prioridad": "alta"
}

// Respuesta 201 Created
{
  "exito": true,
  "datos": {
```

```
    "tarea_id": 15,
    "titulo": "Contrato de interfaz",
    "descripcion": "Documento del servicio de Tareas",
    "usuario_id": 692464,
    "materia_id": 12,
    "grupo_id": 1,
    "fecha_entrega": "2026-09-23",
    "hora_entrega": "18:00",
    "prioridad": "alta",
    "estado": "pendiente",
    "fecha_creacion": "2026-09-16T10:30:00",
    "fecha_actualizacion": "2026-09-16T10:30:00"
  },
  "mensaje": "Tarea creada correctamente"
}
```


| Campo | Tipo | Obligatorio | Descripción |
|---|---|---|---|
| tarea_id | integer | Sí | Identificador único de la tarea. |
| titulo | string (máx. 150) | Sí | Nombre corto de la tarea. |
| descripcion | string (máx. 1000) | No | Detalle de la actividad. |
| usuario_id | integer | Sí | Estudiante responsable. |
| materia_id | integer | Sí | Materia a la que pertenece. |
| grupo_id | integer \| null | No | Grupo de estudio; null si es individual. |
| fecha_entrega | string (YYYY-MM-DD) | Sí | Fecha límite de entrega. |
| hora_entrega | string (HH:MM) | No | Hora límite, formato 24h. |
| prioridad | string | No | baja / media / alta. |
| estado | string | Sí | pendiente / en_progreso / completada / cancelada. |

## H.5 Errores

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 400 Bad Request | Solicitud con datos inválidos o incompletos (falta título, fecha con formato incorrecto, estado no permitido, materia_id inexistente). | Corregir los datos según el mensaje y no reintentar igual. |
| 401 Unauthorized | Token ausente, inválido o vencido. | Pedir al usuario que inicie sesión de nuevo. |
| 404 Not Found | El recurso no existe (tarea_id no registrada). | Informar que la tarea no existe; no reintentar. |
| 500 Internal Server Error | Falla inesperada dentro de Tareas. | Mostrar mensaje genérico y reportar con el X-Request-Id; se puede reintentar una vez. |
| 503 Service Unavailable | Tareas o un servicio del que depende no está disponible. | Mostrar "servicio temporalmente no disponible" y reintentar más tarde. |

Nota del equipo de Tareas: cuando Tareas recibe un 404 de Materias o Usuarios al crear una tarea, responde 400 al consumidor indicando qué referencia no existe, porque el error está en los datos enviados y no en la tarea.

## H.6 Reglas de datos

- Identificadores: integer positivo, nombrados como <entidad>_id (usuario_id, materia_id, grupo_id, tarea_id, notificacion_id).
- Nombres de campos: snake_case, en español y sin tildes (fecha_entrega, no fechaEntrega).
- Fechas: YYYY-MM-DD (ISO 8601). Horas: HH:MM, 24 horas, hora de Colombia (UTC-5). Fecha y hora: YYYY-MM-DDTHH:MM:SS.
- Estados de tarea: pendiente / en_progreso / completada / cancelada (minúsculas). Prioridad: baja / media / alta.
- Textos UTF-8; titulo entre 1 y 150 caracteres; descripcion hasta 1000.
- Listas vacías se devuelven como [] y nunca como null.
- Regla de negocio: fecha_entrega no puede ser anterior a la fecha de creación.

## H.7 Manejo de fallos

Principio del equipo de Tareas: "no hubo respuesta" no significa "no existe la información". Tareas nunca asumirá que un usuario, materia o grupo no existe si el servicio correspondiente no respondió.

| Situación | Comportamiento de Tareas |
|---|---|
| Usuarios o Materias no responden al crear una tarea | Espera máximo 3 segundos, reintenta 2 veces (a 1 y 2 segundos). Si sigue sin respuesta, no crea la tarea, responde 503 y registra el error. |
| Grupos no responde al crear una tarea grupal | Igual que el caso anterior. Las tareas individuales (sin grupo_id) se pueden seguir creando. |
| Notificaciones no responde | La tarea sí se crea (201). El evento queda guardado como pendiente y se reenvía cada 5 minutos hasta por 24 horas. |
| Un servicio falla repetidamente | Después de 5 fallos seguidos, Tareas deja de llamarlo por 30 segundos (circuit breaker) y responde 503 de inmediato. |
| Tareas está caído | Los consumidores muestran "servicio temporalmente no disponible". Estadísticas conserva su último dato y lo marca como desactualizado. |

## H.8 Versionamiento

| Versión | Fecha | Cambio |
|---|---|---|
| 1.0 | 16/09/2026 | Primera definición del contrato (propuesta del equipo antes de la reunión de integración). |
| 1.x | Por definir | Cambios compatibles acordados en la reunión (ej. agregar campos opcionales). |
| 2.0 | Por definir | Cambios que rompan la estructura; se publicarían en /api/v2/ manteniendo /api/v1/ durante la transición. |

## H.9 Prueba de integración

- 1. Crear tarea con usuario, materia y grupo válidos → 201; estado pendiente; Notificaciones recibe tarea_creada.
- 2. Crear tarea con materia_id inexistente → 400 indicando que la materia no existe; no se guarda nada.
- 3. Crear tarea con un usuario que no pertenece al grupo → 400 indicando que el usuario no es integrante del grupo.
- 4. Crear tarea con el servicio Usuarios detenido → 503 tras los reintentos; no se guarda la tarea.

- 5. Crear tarea con Notificaciones detenido → 201; el evento queda pendiente y se envía cuando el servicio vuelva.
- 6. Consultar tarea inexistente → 404.
- 7. Estadísticas consulta /api/v1/tareas/resumen → 200 con los conteos correctos por estado.
- 8. Solicitud sin token → 401.

## H.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const tareaPrioridadSchema = z.enum(["baja", "media", "alta"]);

export const tareaEstadoSchema = z.enum([
  "pendiente",
  "en_progreso",
  "completada",
  "cancelada",
]);

export const tareaSchema = z.object({
  tarea_id: z.number().int().positive(),
  titulo: z.string().min(1).max(150),
  descripcion: z.string().max(1000).optional(),
  usuario_id: z.number().int().positive(),
  materia_id: z.number().int().positive(),
  grupo_id: z.number().int().positive().nullable().optional(),
  fecha_entrega: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  hora_entrega: z.string().regex(/^\d{2}:\d{2}$/).optional(),
  prioridad: tareaPrioridadSchema.optional(),
  estado: tareaEstadoSchema,
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}$/),
  fecha_actualizacion: z.string().regex(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}$/),
});

export type Tarea = z.infer<typeof tareaSchema>;

export const crearTareaInputSchema = z.object({
  titulo: z.string().min(1).max(150),
  descripcion: z.string().max(1000).optional(),
  usuario_id: z.number().int().positive(),
  materia_id: z.number().int().positive(),
  grupo_id: z.number().int().positive().nullable().optional(),
  fecha_entrega: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  hora_entrega: z.string().regex(/^\d{2}:\d{2}$/).optional(),
  prioridad: tareaPrioridadSchema.optional(),
});

export type CrearTareaInput = z.infer<typeof crearTareaInputSchema>;

export const actualizarTareaInputSchema = z.object({
  titulo: z.string().min(1).max(150).optional(),
  descripcion: z.string().max(1000).optional(),
  fecha_entrega: z.string().regex(/^\d{4}-\d{2}-\d{2}$/).optional(),
  hora_entrega: z.string().regex(/^\d{2}:\d{2}$/).optional(),
  prioridad: tareaPrioridadSchema.optional(),
```

```
});

export type ActualizarTareaInput = z.infer<typeof actualizarTareaInputSchema>;

export const actualizarEstadoTareaInputSchema = z.object({
  estado: tareaEstadoSchema,
});

export type ActualizarEstadoTareaInput = z.infer<typeof
actualizarEstadoTareaInputSchema>;

export const tareaListadoItemSchema = z.object({
  tarea_id: z.number().int().positive(),
  titulo: z.string(),
  materia_id: z.number().int().positive(),
  fecha_entrega: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  estado: tareaEstadoSchema,
});

export type TareaListadoItem = z.infer<typeof tareaListadoItemSchema>;

export const tareasListadoSchema = z.object({
  total: z.number().int().nonnegative(),
  pagina: z.number().int().positive(),
  tamano: z.number().int().positive(),
  tareas: z.array(tareaListadoItemSchema),
});

export type TareasListado = z.infer<typeof tareasListadoSchema>;

export const resumenTareasSchema = z.object({
  usuario_id: z.number().int().positive(),
  total: z.number().int().nonnegative(),
  pendientes: z.number().int().nonnegative(),
  en_progreso: z.number().int().nonnegative(),
  completadas: z.number().int().nonnegative(),
  canceladas: z.number().int().nonnegative(),
  vencidas: z.number().int().nonnegative(),
});

export type ResumenTareas = z.infer<typeof resumenTareasSchema>;

export const errorTareasSchema = z.object({
  codigo: z.number().int(),
  tipo: z.string(),
  mensaje: z.string(),
});

export type ErrorTareas = z.infer<typeof errorTareasSchema>;

export function respuestaTareasSchema<T extends z.ZodTypeAny>(datosSchema: T) {
  return z.discriminatedUnion("exito", [
    z.object({ exito: z.literal(true), datos: datosSchema, mensaje: z.string() }),
    z.object({ exito: z.literal(false), error: errorTareasSchema }),
  ]);
}

export const crearTareaResponseSchema = respuestaTareasSchema(tareaSchema);
```

```
export const obtenerTareaResponseSchema = respuestaTareasSchema(tareaSchema);
export const actualizarTareaResponseSchema = respuestaTareasSchema(tareaSchema);
export const listarTareasResponseSchema = respuestaTareasSchema(tareasListadoSchema);
export const cambiarEstadoResponseSchema = respuestaTareasSchema(
  z.object({ tarea_id: z.number().int().positive(), estado: tareaEstadoSchema })
);
export const cancelarTareaResponseSchema = respuestaTareasSchema(
  z.object({ tarea_id: z.number().int().positive(), estado: z.literal("cancelada") })
);
export const resumenTareasResponseSchema = respuestaTareasSchema(resumenTareasSchema);
```

# I. Contrato: estadisticas-service

## I.1 Propósito del servicio

Genera reportes y métricas sobre el desempeño académico y la participación de los usuarios. No es dueño de datos primarios (usuarios, materias, tareas o grupos): consulta esa información a los servicios correspondientes, la procesa y la expone como indicadores agregados. No almacena de forma permanente los datos originales; puede mantener una caché temporal.

## I.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| Datos de usuario | usuarios-service | estadisticas-service |
| Datos de materias | materias-service | estadisticas-service |
| Datos de tareas | tareas-service | estadisticas-service |
| Datos de grupos | grupos-service | estadisticas-service |
| Reporte estadístico | estadisticas-service | Frontend |

## I.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| GET | /api/v1/estadisticas/usuario/{usuario_id} | Reporte individual de desempeño. |
| GET | /api/v1/estadisticas/materia/{materia_id} | Reporte agregado por materia (sin ejemplo de respuesta aún). |
| GET | /api/v1/estadisticas/grupo/{grupo_id} | Reporte de participación por grupo (sin ejemplo de respuesta aún). |

## I.4 Solicitud y respuesta

Parámetros de consulta opcionales para /estadisticas/usuario/{usuario_id}: periodo (string YYYY-MM) y materia_id (integer).

```
// Respuesta 200 OK — GET /api/v1/estadisticas/usuario/25
{
  "usuario_id": 25,
  "nombre": "Ana",
  "periodo": "2026-09",
  "tareas": {
    "total": 20,
    "completadas": 15,
    "pendientes": 3,
    "vencidas": 2,
    "porcentaje_completado": 75.0
  },
  "grupos": {
    "total_grupos": 3,
    "grupos_activos": 2
  },
  "materias": [
    { "materia_id": 4, "nombre": "Sistemas Distribuidos", "tareas_completadas": 5,
"tareas_total": 6 }
  ],
```

```
  "fecha_calculo": "2026-09-16"
}
```

| Campo | Tipo | Obligatorio | Descripción |
|---|---|---|---|
| usuario_id | integer | Sí | Identificador del usuario. |
| nombre | string | Sí | Nombre del estudiante. |
| periodo | string | No | Periodo académico evaluado (YYYY-MM). |
| tareas.porcentaje_completado | number | Sí | Porcentaje de tareas completadas sobre el total. |
| grupos.grupos_activos | integer | Sí | Grupos con actividad reciente. |
| materias | array | Sí | Detalle de avance por materia. |
| fecha_calculo | string (YYYY-MM-DD) | Sí | Fecha en que se generó el reporte. |

## I.5 Errores

| Código | Significado | Comportamiento esperado del consumidor |
|---|---|---|
| 400 | Solicitud incorrecta (parámetro inválido, ej. usuario_id no numérico). | Corregir el parámetro antes de reintentar; no reintentar automáticamente. |
| 404 | Usuario, materia o grupo no encontrado. | Mostrar mensaje de "sin datos disponibles"; no asumir estadísticas en cero. |
| 500 | Error interno del servicio Estadísticas. | Reintentar más tarde; registrar el incidente. |
| 503 | Servicio Estadísticas temporalmente no disponible (dependencia caída). | Mostrar "estadísticas no disponibles por el momento"; reintentar con backoff. |

## I.6 Reglas de datos

- usuario_id / materia_id / tarea_id / grupo_id: entero positivo (siempre sufijo _id, nunca id_usuario).
- Fechas: YYYY-MM-DD. Periodo: YYYY-MM.
- Porcentajes: número decimal entre 0 y 100, con un decimal.
- estado (tareas), según este equipo: pendiente | completada | vencida (ver discrepancia con tareas-service en la sección E).
- Nombres: texto (string), UTF-8.

## I.7 Manejo de fallos

- Si un servicio del que depende (Usuarios, Materias, Tareas o Grupos) no responde: no se asume que el recurso no existe.
- Se devuelve 503 al consumidor, indicando qué dominio de datos no pudo consultarse.
- Se usa, si existe, el último valor en caché (con marca de tiempo), dejando explícito que el dato es "parcial" o "en caché".
- Se registra el error con el servicio afectado, la hora y el recurso solicitado.
- Si Estadísticas es quien falla, el Frontend debe mostrar indisponibilidad temporal sin bloquear el resto de la plataforma.

## I.8 Versionamiento

| Versión | Descripción |
|---|---|
| 1.0 | Primera definición del contrato: endpoints de estadísticas por usuario, materia y grupo. |
| 1.1 (futura) | Posible adición de estadísticas comparativas entre usuarios de un mismo grupo. |
| 2.0 (futura) | Posible cambio de estructura de respuesta si se agregan nuevas dimensiones de análisis. |

## I.9 Prueba de integración

- 1. Consultar GET /api/v1/estadisticas/usuario/{id} con un usuario real y verificar que el nombre coincida con usuarios-service.
- 2. Verificar que el conteo de tareas completadas/pendientes coincide con lo devuelto directamente por tareas-service para ese usuario.
- 3. Apagar temporalmente grupos-service y confirmar que Estadísticas responde 503 (o datos en caché marcados como tales), no ceros.
- 4. Verificar formatos: fechas YYYY-MM-DD, ids como enteros positivos, porcentajes como decimales.

## I.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const consultarEstadisticasUsuarioQuerySchema = z.object({
  periodo: z.string().regex(/^\d{4}-\d{2}$/).optional(),
  materia_id: z.number().int().positive().optional(),
});

export type ConsultarEstadisticasUsuarioQuery = z.infer<typeof
consultarEstadisticasUsuarioQuerySchema>;

export const tareasResumenSchema = z.object({
  total: z.number().int().nonnegative(),
  completadas: z.number().int().nonnegative(),
  pendientes: z.number().int().nonnegative(),
  vencidas: z.number().int().nonnegative(),
  porcentaje_completado: z.number().min(0).max(100),
});

export type TareasResumen = z.infer<typeof tareasResumenSchema>;

export const gruposResumenSchema = z.object({
  total_grupos: z.number().int().nonnegative(),
  grupos_activos: z.number().int().nonnegative(),
});

export type GruposResumen = z.infer<typeof gruposResumenSchema>;

export const materiaResumenSchema = z.object({
  materia_id: z.number().int().positive(),
  nombre: z.string(),
  tareas_completadas: z.number().int().nonnegative(),
  tareas_total: z.number().int().nonnegative(),
});
```

```
export type MateriaResumen = z.infer<typeof materiaResumenSchema>;

export const estadisticasUsuarioSchema = z.object({
  usuario_id: z.number().int().positive(),
  nombre: z.string(),
  periodo: z.string().regex(/^\d{4}-\d{2}$/).optional(),
  tareas: tareasResumenSchema,
  grupos: gruposResumenSchema,
  materias: z.array(materiaResumenSchema),
  fecha_calculo: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
});

export type EstadisticasUsuario = z.infer<typeof estadisticasUsuarioSchema>;
```

# J. Contrato: grupos-service

*Nota de procedencia: a diferencia de los cuatro servicios anteriores, de Grupos solo se recibió la forma de los datos (JSON Schema), sin secciones de errores, reglas de fallo, versionamiento ni pruebas. Esas partes de esta sección son una PROPUESTA del equipo Frontend, no una confirmación de Grupos, y quedan marcadas como tal.*

## J.1 Propósito del servicio

Administra los grupos de trabajo que los estudiantes conforman dentro de una materia, y sus integrantes, para permitir la colaboración en tareas grupales.

## J.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| CRUD de grupos | grupos-service | Frontend |
| Validar grupo e integrantes para una tarea | grupos-service | tareas-service |
| Grupos y actividad del usuario | grupos-service | estadisticas-service |

## J.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| POST | /api/v1/grupos | Crear un grupo. |
| GET | /api/v1/grupos | Listar mis grupos. |
| GET | /api/v1/grupos/{grupo_id} | Consultar un grupo — usado también por tareas-service. |
| POST | /api/v1/grupos/{grupo_id}/integrantes | Invitar un integrante. |

## J.4 Solicitud y respuesta

```
// Solicitud: POST /api/v1/grupos
{
  "nombre": "Grupo de estudio - Sistemas Distribuidos",
  "materia_id": 12
}

// Respuesta 201 Created
{
  "grupo_id": 7,
  "nombre": "Grupo de estudio - Sistemas Distribuidos",
  "materia_id": 12,
  "creado_por": 692464,
  "integrantes": [
    { "usuario_id": 692464, "rol": "lider", "unido_en": "2026-09-20T14:30:00" }
  ],
  "fecha_creacion": "2026-09-20T14:30:00"
}
```

| Campo | Tipo | Obligatorio | Descripción |
|---|---|---|---|
| grupo_id | integer | Sí | Identificador único del grupo. |

| | | | |
|---|---|---|---|
| nombre | string | Sí | Nombre del grupo. |
| materia_id | integer | Sí | Materia a la que pertenece el grupo. |
| creado_por | integer | Sí | usuario_id de quien creó el grupo. |
| integrantes | array | Sí | Lista de {usuario_id, rol, unido_en}. |
| fecha_creacion | string (YYYY-MM-DDTHH:MM:SS) | Sí | Momento de creación, hora Colombia. |

## J.5 Errores (propuesta del Frontend, no confirmada por Grupos)

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 400 | nombre o materia_id ausente o inválido. | Corregir el valor antes de reintentar. |
| 401 | Token ausente o inválido. | Redirigir a login. |
| 404 | El grupo no existe, o el usuario no pertenece a él. | No asumir que el id es inválido en general. |
| 500 | Error interno de grupos-service. | Registrar el error; reintentar una vez. |
| 503 | grupos-service temporalmente no disponible. | No asumir que el grupo no existe; no crear la tarea grupal dependiente hasta confirmar. |

## J.6 Reglas de datos

- Identificadores (grupo_id, materia_id, creado_por, usuario_id): enteros positivos.
- rol del integrante: lider / miembro.
- Fechas (fecha_creacion, unido_en): YYYY-MM-DDTHH:MM:SS, hora de Colombia — misma convención que Tareas y Materias.
- Convención de nombres: snake_case.

## J.7 Manejo de fallos (propuesta del Frontend, no confirmada por Grupos)

- Si grupos-service no responde a tareas-service al crear una tarea grupal: no asumir que el grupo no existe, responder 503, no crear la tarea.
- Las tareas individuales (sin grupo_id) no deben verse afectadas por una caída de grupos-service.

## J.8 Versionamiento

No confirmado por el equipo de Grupos. Se asume 1.0 sobre /api/v1/ hasta que lo definan explícitamente.

## J.9 Prueba de integración (propuesta)

- 1. Crear un grupo desde el Frontend y confirmar 201 con los campos documentados.
- 2. tareas-service consulta GET /api/v1/grupos/{grupo_id} con ese grupo real y confirma 200, validando que el usuario pertenece a integrantes.
- 3. Repetir con un grupo inexistente → 404.
- 4. Detener grupos-service y confirmar que tareas-service responde 503 al crear una tarea grupal, sin afectar tareas individuales.

## J.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const integranteRolSchema = z.enum(["lider", "miembro"]);

export const integranteSchema = z.object({
  usuario_id: z.number().int().positive(),
  rol: integranteRolSchema,
  unido_en: z.string().regex(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}$/),
});

export type Integrante = z.infer<typeof integranteSchema>;

export const grupoSchema = z.object({
  grupo_id: z.number().int().positive(),
  nombre: z.string().min(1),
  materia_id: z.number().int().positive(),
  creado_por: z.number().int().positive(),
  integrantes: z.array(integranteSchema),
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}$/),
});

export type Grupo = z.infer<typeof grupoSchema>;

export const crearGrupoInputSchema = z.object({
  nombre: z.string().min(1),
  materia_id: z.number().int().positive(),
});

export type CrearGrupoInput = z.infer<typeof crearGrupoInputSchema>;

export const invitarIntegranteInputSchema = z.object({
  usuario_id: z.number().int().positive(),
});

export type InvitarIntegranteInput = z.infer<typeof invitarIntegranteInputSchema>;

export const gruposListadoSchema = z.array(grupoSchema);

export const mensajeExitoSchema = z.object({
  mensaje: z.string(),
});

export type MensajeExito = z.infer<typeof mensajeExitoSchema>;
```

# K. Contrato: notificaciones-service

## K.1 Propósito del servicio

Gestiona y registra las notificaciones que reciben los estudiantes desde los demás servicios: próximas fechas de entrega, creación o actualización de tareas, recordatorios académicos y avisos importantes. Centraliza los avisos de los demás servicios, garantizando trazabilidad y consistencia para el usuario final.

## K.2 Proveedor y consumidor

| Integración | Proveedor | Consumidor |
|---|---|---|
| Validación del estudiante | usuarios-service | notificaciones-service |
| Tareas pendientes del usuario | tareas-service | notificaciones-service |
| Grupos a los que pertenece el usuario | grupos-service | notificaciones-service |
| Materias del usuario | materias-service | notificaciones-service |
| Notificaciones del usuario | notificaciones-service | Frontend |

## K.3 Operaciones

| Método | Endpoint | Propósito |
|---|---|---|
| POST | /api/v1/notification/create-notifications | Usado por Usuarios, Tareas, Grupos y Materias para registrar un nuevo aviso (no lo llama el Frontend). |
| GET | /api/v1/notification/{usuario_id} | Expone las notificaciones del usuario consultado — este sí lo consume el Frontend. |

## K.4 Solicitud y respuesta

### Crear notificación (llamado por otros backends)

```
// Solicitud: POST /api/v1/notification/create-notifications
{
  "usuario_id": 25,
  "tipo": "recordatorio",
  "titulo": "Nueva acción asignada",
  "mensaje": "Se te ha asignado una nueva acción.",
  "fecha_creacion": "2026-09-22",
  "estado": "realizado",
  "referencia_id": 550
}

// Respuesta 201 Created
{
  "codigo": 201,
  "mensaje": "Notificación creada correctamente",
  "fecha_creacion": "2026-09-22"
}
```

### Listar notificaciones del usuario (consumido por el Frontend)

*El ejemplo de respuesta es tal como aparece en el documento de origen — nótese que no incluye notificacion_id, usuario_id, fecha_creacion ni referencia_id, aunque la tabla de datos de ese mismo documento los define como parte de la notificación (ver discrepancia en la sección E).*

```
// Respuesta 200 OK — GET /api/v1/notification/{usuario_id}
{
  "tipo": "recordatorio",
  "titulo": "Nueva acción asignada",
  "mensaje": "Se te ha asignado una nueva acción.",
  "estado": "realizado"
}
```

| Campo | Tipo | Obligatorio | Descripción (según sección D del contrato) |
|---|---|---|---|
| notificacion_id | integer | Sí | Identificador de registro. |
| usuario_id | integer | Sí | Identificador de usuario. |
| tipo | string | Sí | Origen/categoría de la notificación (sin lista cerrada de valores). |
| titulo | string | Sí | Título de la acción. |
| mensaje | string | Sí | Descripción de la acción. |
| fecha_creacion | string (YYYY-MM-DD) | Sí | Fecha en que se generó. |
| estado | string | Sí | Ver contradicción en la sección E. |
| referencia_id | integer | No | Id del elemento que originó la notificación (tarea, grupo o materia). |

## K.5 Errores

*Crear notificación:*

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 400 | Body inválido, campos obligatorios ausentes. | Enviar el body completo según el ejemplo. |
| 500 | Fallo inesperado en el servidor, base de datos o servicio interno al crear la notificación. | Reintentar (el documento de origen dice "remontar el contenedor"). |
| 503 | El servicio de notificaciones está caído. | Reintentar más tarde. |

*Listar notificaciones:*

| Código | Significado | Qué debe hacer el consumidor |
|---|---|---|
| 404 | El usuario_id consultado no existe. | Consultar un id válido. |
| 500 | Error inesperado del servicio. | Reintentar. |
| 503 | El servicio de notificaciones está caído. | Reintentar más tarde. |

## K.6 Reglas de datos

- Formato de comunicación: JSON.
- Identificadores: notificacion_id, usuario_id, tarea_id, grupo_id, materia_id — todos enteros.
- Fecha: YYYY-MM-DD. Hora: HH:MM (regla declarada por el equipo, aunque ningún ejemplo la usa — ver discrepancia).

- API base: /api/v1/notification (nótese: singular, distinto del /api/v1/notificaciones que se podría esperar por analogía con los demás servicios).

## K.7 Manejo de fallos

*El documento de origen no incluye un procedimiento de fallos ante caída de servicios externos (como sí lo hacen Tareas y Estadísticas) — solo define qué código HTTP devolver. Comportamiento mínimo recomendado, pendiente de confirmar con el equipo:*

- Si notificaciones-service no responde al Frontend: mostrar centro de notificaciones vacío con aviso de "no disponible", no como si el usuario no tuviera notificaciones.
- Si notificaciones-service no responde a otro backend que intenta crear un aviso (ej. tareas-service): ese backend no debe fallar su propia operación principal por esto — la tarea igual se crea, y el aviso queda pendiente de reintento (así lo maneja Tareas del lado suyo).

## K.8 Versionamiento

| Versión | Fecha | Cambio |
|---|---|---|
| 1.0 | 24/09/2026 | Primera propuesta. |
| 1.1 | Por definir | Por definir. |
| 1.2 | Por definir | Por definir. |

## K.9 Prueba de integración (propuesta del Frontend — el documento de origen no definió pruebas)

- 1. tareas-service crea una tarea próxima a vencer → confirmar que llega una notificación tipo recordatorio para ese usuario.
- 2. GET /api/v1/notification/{usuario_id} con un usuario real → confirmar que devuelve un arreglo con todos los campos de la sección D, no solo los cuatro del ejemplo del documento.
- 3. GET con un usuario_id inexistente → 404.
- 4. Detener notificaciones-service y confirmar que tareas-service sigue creando tareas con normalidad (el aviso queda pendiente, no bloquea la operación principal).

## K.10 Contrato de datos (Zod)

```
import { z } from "zod";

export const notificacionSchema = z.object({
  notificacion_id: z.number().int().positive(),
  usuario_id: z.number().int().positive(),
  tipo: z.string(),
  titulo: z.string().min(1),
  mensaje: z.string().min(1),
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  estado: z.string(),
  referencia_id: z.number().int().positive().optional(),
});

export type Notificacion = z.infer<typeof notificacionSchema>;

export const notificacionesListadoSchema = z.array(notificacionSchema);
```

```
export const crearNotificacionInputSchema = z.object({
  usuario_id: z.number().int().positive(),
  tipo: z.string(),
  titulo: z.string().min(1),
  mensaje: z.string().min(1),
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  estado: z.string(),
  referencia_id: z.number().int().positive().optional(),
});

export type CrearNotificacionInput = z.infer<typeof crearNotificacionInputSchema>;

export const crearNotificacionResponseSchema = z.object({
  codigo: z.literal(201),
  mensaje: z.string(),
  fecha_creacion: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
});

export type CrearNotificacionResponse = z.infer<typeof crearNotificacionResponseSchema>;

export const errorNotificacionesSchema = z.object({
  codigo: z.number().int(),
  mensaje: z.string(),
  detalle: z.string().optional(),
});

export type ErrorNotificaciones = z.infer<typeof errorNotificacionesSchema>;
```

## L. Estado final de la integración

Con esta entrega, los seis microservicios de CampusStudy quedan documentados. Antes de programar cualquier integración real, deben resolverse en la reunión los puntos listados en la sección E — en particular, las dos contradicciones internas del contrato de Notificaciones y la confirmación formal del contrato de Grupos, que hasta ahora se trató como propuesta.